1. Estrategia de pruebas

Las pruebas del Sprint 2 tienen como objetivo verificar el correcto funcionamiento del backend y la integración del Módulo 1 del sistema, considerando las funcionalidades desarrolladas para la gestión de clientes, direcciones, productos, inventario, movimientos de stock, usuarios, roles, autenticación y proveedores. Asimismo, se evalúa el comportamiento del sistema bajo condiciones de carga definidas y se verifica la aplicación de medidas de seguridad para proteger el acceso a las funcionalidades y los datos.

El alcance de las pruebas corresponde a las historias de usuario trabajadas durante el Sprint 2:

- **HU-01 a HU-05:** Gestión de clientes y direcciones.
- **HU-06 a HU-09:** Gestión de productos e inventario.
- **HU-10:** Registro de movimientos de inventario.
- **HU-11 y HU-12:** Gestión de usuarios, roles y autenticación.
- **HU-26 a HU-29:** Gestión de proveedores.

Las pruebas considerarán cuatro categorías:

- **PU — Pruebas unitarias:** validación del comportamiento de funciones, métodos o componentes aislados del backend.
- **PI — Pruebas de integración:** validación de la comunicación entre los componentes del sistema, principalmente frontend, API desarrollada en Django y base de datos.
- **PR — Pruebas de rendimiento:** evaluación de los tiempos de respuesta, volumen de solicitudes, solicitudes fallidas y excepciones de operaciones representativas, utilizando Locust bajo condiciones de carga definidas.
- **PS — Pruebas de seguridad:** validación del acceso sin autenticación, la autorización según roles y permisos, la validación de entradas, la protección de credenciales, el rechazo de roles inválidos, la validación de tokens y la resistencia ante intentos de manipulación de privilegios. También se verifica que las operaciones no autorizadas no provoquen modificaciones en los datos del sistema.

En materia de seguridad, se implementó autenticación mediante tokens Bearer con firma y expiración, junto con mecanismos de autorización basados en el usuario autenticado y los permisos asignados. Estas medidas se evaluaron mediante pruebas de acceso no autenticado, autorización por roles, validación de tokens, manipulación de encabezados y protección de operaciones sobre clientes e inventario.

Quedan fuera del alcance de este Sprint las funcionalidades de cotizaciones, órdenes de compra, facturación, notas de crédito, generación y envío de documentos comerciales y modelos predictivos de inteligencia artificial, debido a que corresponden a Sprints posteriores.

Herramientas utilizadas

- **Django y Django REST Framework:** desarrollo y exposición de las API del backend, validación de solicitudes y aplicación de mecanismos de autenticación y autorización.
- **Supabase:** servicio de base de datos utilizado por el sistema.
- **Docker:** ejecución del ambiente del backend.
- **Navegador web:** validación funcional de las funcionalidades integradas con el frontend.
- **Postman:** ejecución de solicitudes HTTP para verificar el comportamiento de los endpoints, los códigos de respuesta y las restricciones de seguridad.
- **Framework de pruebas de Django:** ejecución de pruebas automatizadas unitarias.
- **Locust:** simulación de usuarios concurrentes y evaluación del rendimiento de las API, con generación de estadísticas, reportes y archivos de resultados.
- **Git y GitHub:** control de versiones y registro de los cambios realizados en el proyecto.

Ambiente de pruebas

Las pruebas se realizaron sobre el backend desarrollado en Django REST Framework, utilizando Supabase como servicio de base de datos y Docker para la ejecución del ambiente del backend. Las pruebas de integración consideraron la comunicación entre el frontend, las API y la base de datos, mientras que las pruebas de seguridad incluyeron solicitudes directas a los endpoints mediante Postman.

Para las pruebas de rendimiento se utilizó Locust, configurado para simular hasta 10 usuarios concurrentes, con una tasa de incorporación de 2 usuarios por segundo y una pausa de 1 segundo entre tareas de cada usuario. Los resultados se analizaron considerando las solicitudes procesadas, los tiempos de respuesta, las solicitudes fallidas, las excepciones y los criterios de aceptación establecidos, incluido el límite de 3 segundos definido en el RNF-11.

Las pruebas se efectuaron utilizando datos de prueba controlados, procurando evitar la exposición de información real o sensible. Los resultados y las evidencias se conservaron mediante capturas de pantalla, reportes y archivos generados por las herramientas utilizadas, identificados según la nomenclatura correspondiente a cada categoría de prueba.

El alcance de la evaluación corresponde a las funcionalidades del Sprint 2 y a la versión del sistema sometida a pruebas, sin incluir las funcionalidades planificadas para Sprints posteriores.

2. Pruebas unitarias — PU

Las pruebas unitarias se enfocan en verificar el funcionamiento de unidades específicas del backend, incluyendo funciones de validación y normalización, métodos de los serializers, campos personalizados y cálculos asociados a la lógica de negocio.

Se utilizarán pruebas automatizadas como evidencia del funcionamiento del código, evaluando las salidas obtenidas frente a los resultados esperados para distintas entradas. Entre los comportamientos evaluados se incluyen la normalización de RUT, la conversión de roles de usuario a sus identificadores, la generación automática de códigos de lote y el cálculo del stock de productos.

Las pruebas se implementan mediante `SimpleTestCase`, lo que permite comprobar estos comportamientos sin acceder directamente a la base de datos. En los casos que requieren simular operaciones de persistencia o consultas, se utilizan mecanismos de simulación mediante `unittest.mock.patch`.

Importante: *La clasificación dependerá del comportamiento que se verifique. Las pruebas que evalúan directamente funciones o métodos específicos se registrarán como pruebas unitarias (PU), mientras que aquellas que comprueban la interacción entre componentes mediante solicitudes a los endpoints se considerarán pruebas de integración (PI). Ambos tipos pueden ejecutarse de forma automatizada, pero se diferencian por su objetivo y alcance.*

ID	PU-01 – Normalizar rut chileno válido
Unidad probada	`normalize_rut()` / validación de RUT
Entrada	RUT chileno válido con formato `11.111.111-1`
Resultado esperado	El RUT es validado y normalizado a `11111111-1`
Resultado obtenido	La función normalizó correctamente el RUT de entrada y devolvió 11111111-1, coincidiendo con el resultado esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_01_normaliza_rut_chileno_valido`” del archivo “`clientes/test_unitarias_cliente.py`” Evidencia documentada: “`PU-01-01-ResultadosTerminal.png`” y “`PU-01-02-TestUnitariasCliente.py`”

ID	PU-02 – Rechazar rut con dígito verificador incorrecto

Unidad probada	`normalize_rut()` / validación de RUT
Entrada	RUT con dígito verificador incorrecto
Resultado esperado	Se rechaza el RUT mediante `ValidationError`
Resultado obtenido	La función lanzó la excepción de validación esperada, por lo que el RUT inválido fue rechazado correctamente.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: "`test_pu_02_rechaza_rut_con_digito_verificador_incorrecto`" del archivo "`clientes/test_unitarias_cliente.py`" Evidencia documentada: "`PU-01-01-ResultadosTerminal.png`" y "`PU-01-02-TestUnitariasCliente.py`"

ID	PU-03 – Convierte EstadoCliente activo a true
Unidad probada	`EstadoClienteField.to_internal_value()`
Entrada	`"Activo"`
Resultado esperado	Se transforma correctamente a booleano `True`
Resultado obtenido	El método devolvió `True`, coincidiendo con el resultado esperado.
Estado	Aprobada.
Evidencia	Ejecución automatizada del método: "`test_pu_03_convierte_activo_a_true`" del archivo "`clientes/test_unitarias_cliente.py`" Evidencia documentada: "`PU-01-01-ResultadosTerminal.png`" y "`PU-01-02-TestUnitariasCliente.py`"

ID	PU-04 – Convierte EstadoCliente inactivo a false
Unidad probada	`EstadoClienteField.to_internal_value()`
Entrada	`"Inactivo"`
Resultado esperado	Se transforma correctamente a booleano `False`

Resultado obtenido	El método devolvió `False`, coincidiendo con el resultado esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_04_convierte_inactivo_a_false`” del archivo “`clientes/test_unitarias_cliente.py`” Evidencia documentada: “`PU-01-01-ResultadosTerminal.png`” y “`PU-01-02-TestUnitariasCliente.py`”

ID	PU-05 – EstadoCliente rechaza estado no permitido
Unidad probada	`EstadoClienteField.to_internal_value()`
Entrada	`"Pendiente"`
Resultado esperado	Se rechaza mediante `ValidationError`
Resultado obtenido	La función lanzó la excepción de validación esperada, por lo que el estado inválido fue rechazado correctamente.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_05_rechaza_estado_no_permitido`” del archivo “`clientes/test_unitarias_cliente.py`” Evidencia documentada: “`PU-01-01-ResultadosTerminal.png`” y “`PU-01-02-TestUnitariasCliente.py`”

ID	PU-06 — EstadoCliente acepta booleanos como entrada
Unidad probada	`EstadoClienteField.to_internal_value()`
Entrada	`True` y `False`
Resultado esperado	Se aceptan los valores booleanos.
Resultado obtenido	Se aceptaron los valores booleanos como esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_06_acepta_booleanos_como_entrada`” del archivo “`clientes/test_unitarias_cliente.py`”

	Evidencia documentada: “PU-01-01-ResultadosTerminal.png” y “PU-01-02-TestUnitariasCliente.py”

ID	PU-07– EstadoCliente representa estado activo como texto
Unidad probada	`EstadoClienteField.to_representation()`
Entrada	`True`
Resultado esperado	Devuelve `"Activo"`
Resultado obtenido	El método devolvió `"Activo"`, coincidiendo con el resultado esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_07_representa_estado_activo_como_texto`” del archivo “`clientes/test_unitarias_cliente.py`” Evidencia documentada: “PU-01-01-ResultadosTerminal.png” y “PU-01-02-TestUnitariasCliente.py”

ID	PU-08 – EstadoCliente representa estado inactivo como texto
Unidad probada	`EstadoClienteField.to_representation()`
Entrada	`False`
Resultado esperado	Devuelve `"Inactivo"`
Resultado obtenido	El método devolvió `False`, coincidiendo con el resultado esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: “`test_pu_08_representa_estado_inactivo_como_texto`” del archivo “`clientes/test_unitarias_cliente.py`” Evidencia documentada: “PU-01-01-ResultadosTerminal.png” y “PU-01-02-TestUnitariasCliente.py”

ID	PU-09 – RolUsuario convierte superusuario admin a id

Unidad probada	`RolUsuarioField.to_internal_value()`
Entrada	`"Superusuario / Admin"`
Resultado esperado	Devuelve el identificador `1`
Resultado obtenido	El campo transformó correctamente el valor `"Superusuario / Admin"` al identificador `1`, coincidiendo con el resultado esperado.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: `test_pu_09_convierte_superusuario_admin_a_id`, perteneciente al archivo `usuarios/test_unitarias_usuario.py`. Evidencia documentada: “`PU-09-01-ResultadosTerminal.png`” y “`PU-09-02-TestUnitariasUsuario.py`”

ID	PU-10 – RolUsuario rechaza rol inexistente
Unidad probada	`RolUsuarioField.to_internal_value()`
Entrada	`"Rol inexistente"`
Resultado esperado	Lanza `serializers.ValidationError`
Resultado obtenido	La entrada es rechazada mediante `ValidationError`, indicando que el rol no es válido.
Estado	Aprobada
Evidencia	Ejecución automatizada del método: `test_pu_10_rechaza_rol_inexistente`, perteneciente al archivo `usuarios/test_unitarias_usuario.py`. Evidencia documentada: “`PU-09-01-ResultadosTerminal.png`” y “`PU-09-02-TestUnitariasUsuario.py`”

ID	PU-11 – EstadoUsuario convierte activo a true e inactivo a false
Unidad probada	`EstadoUsuarioField.to_internal_value()`
Entrada	`"Activo"` e `"Inactivo"`
Resultado esperado	El campo transforma `"Activo"` en el valor booleano `True` e `"Inactivo"` en el valor booleano `False`.

Resultado obtenido	Ambas conversiones se realizaron correctamente. El valor `"Activo"` fue transformado a `True` y el valor `"Inactivo"` a `False`, coincidiendo con los resultados esperados.
Estado	Aprobada
Evidencia	Ejecución automatizada de los métodos: `test_pu_11_convierte_activo_a_true` y `test_pu_11_convierte_inactivo_a_false`, pertenecientes al archivo `usuarios/test_unitarias_usuario.py` Evidencia documentada: “`PU-09-01-ResultadosTerminal.png`” y “`PU-09-02-TestUnitariasUsuario.py`”

ID	PU-12 – Campo permiso acepta objeto json válido
Unidad probada	`UsuarioSerializer.validate_permisos()`

Entrada	Objeto JSON válido:
	`{`
	`"clientes": ["crear", "consultar"],`
	`"inventario": ["consultar"],`
	`}`

Resultado esperado	El método acepta el objeto recibido y devuelve los permisos sin modificaciones.
Resultado obtenido	El método aceptó el diccionario de permisos y devolvió el mismo contenido recibido, coincidiendo con el resultado esperado.
Estado	Aprobado
Evidencia	Archivo `usuarios/test_unitarias_usuario.py`, método `test_pu_12_acepta_objeto_json_valido` Evidencia documentada: “`PU-09-01-ResultadosTerminal.png`” y “`PU-09-02-TestUnitariasUsuario.py`”

ID	PU-13 – Campo permiso rechaza valor que no es objeto json
Unidad probada	`UsuarioSerializer.validate_permisos()`
Entrada	Valor que no corresponde a un objeto JSON: `["crear", "consultar"]`
Resultado esperado	La entrada es rechazada mediante `serializers.ValidationError`
Resultado obtenido	El método rechazó la lista recibida y generó la excepción de validación esperada.

Estado	Aprobada
Evidencia	Archivo `usuarios/test_unitarias_usuario.py`, método `test_pu_13_rechaza_valor_que_no_es_objeto_json` Evidencia documentada: “`PU-09-01-ResultadosTerminal.png`” y “`PU-09-02-TestUnitariasUsuario.py`”

ID	PU-14 – Validación de estados permitidos para proveedores
Unidad probada	`ProveedorSerializer.validate_estado()`
Entrada	`"Activo"` e `"Inactivo"`
Resultado esperado	El método acepta ambos valores y los devuelve sin modificaciones.
Resultado obtenido	Los valores `"Activo"` e `"Inactivo"` fueron aceptados correctamente y devueltos sin modificaciones, coincidiendo con los resultados esperados.
Estado	Aprobado
Evidencia	Archivo `proveedores/test_unitarias_proveedor.py`, métodos `test_pu_14_acepta_estado_activo` y `test_pu_14_acepta_estado_inactivo` Evidencia documentada: “`PU-14-01-ResultadosTerminal.png`” y “`PU-14-02-TestUnitariasProveedor.py`”

ID	PU-15 – Rechazo de estados no permitidos para proveedores
Unidad probada	`ProveedorSerializer.validate_estado()`
Entrada	`"Pendiente"`
Resultado esperado	El método rechaza el valor mediante `serializers.ValidationError`
Resultado obtenido	El valor `"Pendiente"` fue rechazado correctamente, coincidiendo con el resultado esperado.
Estado	Aprobado
Evidencia	Archivo `proveedores/test_unitarias_proveedor.py`, método `test_pu_15_rechaza_estado_no_permitido` Evidencia documentada: “`PU-14-01-ResultadosTerminal.png`” y “`PU-14-02-TestUnitariasProveedor.py`”

ID	PU-16 – Generación automática del código de lote
Unidad probada	`InventarioSerializer.create()`

Entrada	Datos de un lote de inventario sin informar el campo codigo_lote:
	`{`
	`"id_producto": 1,`
	`"cantidad": Decimal("25.00"),`
	`"tipo_envase": "Bidón",`
	`"fecha_fabricacion": "2026-01-01",`
	`"fecha_vencimiento": "2027-01-01",`
	`"estado": "Disponible",`
	`}`

Resultado esperado	Se genera automáticamente un código `LOTE-XXXX`, donde `XXXX` corresponde a cuatro dígitos numéricos.
Resultado obtenido	El método genera un código que cumple el formato establecido, compuesto por el prefijo `LOTE-` seguido de cuatro dígitos numéricos.
Estado	Aprobado
Evidencia	Ejecución del método `test_pu_16_genera_codigo_lote_automaticamente`, ubicado en `inventario/test_unitarias_inventario.py` Evidencia documentada: “`PU-16-01-ResultadosTerminal.png`” y “`PU-16-02-TestUnitariasInventario.py`”

ID	PU-17 – Cálculo del stock de un producto
Unidad probada	`ProductoSerializer.get_stock()`
Entrada	Producto asociado a registros de inventario cuya cantidad total es de 35,50 unidades.
Resultado esperado	Se calcula la suma de las cantidades de los registros de inventario asociados al producto y se devuelve como valor numérico.
Resultado obtenido	La prueba automatizada verificó que el método devuelve el valor 35.5, coincidiendo con la cantidad total proporcionada mediante la simulación de la consulta.
Estado	Aprobado
Evidencia	Ejecución del método `test_pu_17_calcula_stock_de_varios_lotes`, ubicado en `inventario/test_unitarias_inventario.py` Evidencia documentada:

	“PU-16-01-ResultadosTerminal.png” y “PU-16-02-TestUnitariasInventario.py”

ID	PU-18 — Verificación del stock cuando no existen cantidades registradas
Unidad probada	`ProductoSerializer.get_stock()`
Entrada	Producto sin cantidades de inventario registradas. La consulta de inventario devuelve `None` como resultado de la agregación.
Resultado esperado	El método devuelve el valor numérico `0.0`, evitando que la ausencia de registros genere un error o un valor nulo en el stock del producto.
Resultado obtenido	El método devolvió `0.0` cuando la consulta simulada no proporcionó una cantidad total, coincidiendo con el resultado esperado.
Estado	Aprobado
Evidencia	Ejecución automatizada del método `test_pu_18_devuelve_cero_si_no_hay_stock`, ubicado en `inventario/test_unitarias_inventario.py` Evidencia documentada: “PU-16-01-ResultadosTerminal.png” y “PU-16-02-TestUnitariasInventario.py”

Evidencia esperada

Para cada prueba unitaria se conservará, según corresponda:

- Capturas de los resultados de ejecución de las pruebas automatizadas.
- Archivos fuente de las pruebas unitarias implementadas.

3. Pruebas de integración — PI

Las pruebas de integración constituyen la principal evidencia del Sprint 2, debido a que el objetivo de S2-T06 fue integrar y probar el Módulo 1.

Las pruebas se ejecutaron principalmente mediante la interacción con el frontend, verificando el comportamiento de las funcionalidades implementadas y las respuestas de la API desarrollada con Django REST Framework. El flujo general considerado fue:

Frontend → API Django → Supabase → respuesta del backend → frontend.

Cuando corresponda, también se comprobará que los datos enviados mediante la API sean almacenados o modificados correctamente en la base de datos.

3.1 Clientes y direcciones

ID	PI-01 — Registrar y consultar cliente
HU asociadas	HU-01, HU-02.
Objetivo	Verificar que un usuario pueda registrar un cliente y posteriormente consultar su información.
Procedimiento	**1.** Ingresar al módulo de clientes. **2.** Seleccionar la opción de registrar cliente. **3.** Ingresar datos válidos. **4.** Guardar el cliente. **5.** Consultar el listado de clientes. **6.** Verificar que el cliente registrado aparezca correctamente.
Resultado esperado	El cliente es almacenado correctamente y posteriormente aparece disponible para consulta.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se registró un cliente con datos válidos desde el frontend y se verificó que el registro apareciera correctamente en el listado de clientes. Además, se confirmó la persistencia de los datos en Supabase.
Evidencia	`PI-01-01`, `PI-01-02`, `PI-01-03`, `PI-01-04`
Conclusión	**Aprobada**, ya que se verificó el registro y la consulta de clientes desde el frontend, junto con la persistencia de la información en Supabase. Asimismo, la respuesta `HTTP 201 Created` respalda que la solicitud de creación fue procesada correctamente, validando el funcionamiento del flujo de integración para las operaciones probadas.

ID	PI-02 — Modificar cliente
HU asociadas	HU-03.
Objetivo	Verificar que los datos de un cliente existente puedan ser modificados.

Procedimiento	1. Seleccionar un cliente existente. 2. Modificar uno o más datos. 3. Guardar los cambios. 4. Consultar nuevamente el cliente. 5. Verificar que los nuevos datos permanezcan almacenados.
Resultado esperado	La información del cliente se actualiza correctamente.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se modificaron los datos de un cliente existente desde el frontend y se verificó que los cambios se reflejaran al consultar nuevamente su información. Además, se confirmó la actualización de los datos en Supabase cuando correspondía.
Evidencia	`PI-02-01`, `PI-02-02`, `PI-02-03`, `PI-02-04`, `PI-02-05`
Conclusión	**Aprobada**, ya que se verificó que los datos de un cliente existente pueden modificarse desde el frontend y que los cambios se mantienen almacenados correctamente, validando el funcionamiento de la operación de actualización del módulo de clientes.

ID	PI-03 — Registrar, consultar y modificar direcciones
HU asociadas	HU-04, HU-05.
Objetivo	Verificar que un cliente pueda tener múltiples direcciones y que estas puedan ser consultadas y modificadas.
Procedimiento	1. Seleccionar un cliente. 2. Registrar una primera dirección. 3. Registrar una segunda dirección. 4. Consultar las direcciones asociadas. 5. Modificar una de ellas. 6. Verificar que el cambio se encuentre almacenado.
Resultado esperado	El cliente puede mantener múltiples direcciones y estas pueden ser consultadas y modificadas correctamente.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se registraron dos direcciones asociadas a un mismo cliente y se verificó que ambas se visualizaran correctamente en el frontend y quedaran almacenadas en Supabase. Posteriormente, se modificó una de las direcciones y se comprobó que los cambios se reflejaran correctamente tanto en la interfaz como en la base de datos.
Evidencia	`PI-03-01`, `PI-03-02`, `PI-03-03`, `PI-03-04`, `PI-03-05`, `PI-03-06`, `PI-03-07`, `PI-03-08`, `PI-03-09`, `PI-03-10`.
Conclusión	**Aprobada**, ya que se verificó que un cliente puede mantener múltiples direcciones, consultarlas y modificar sus datos correctamente. Además, se confirmó la persistencia de la información y sus actualizaciones en Supabase, validando el funcionamiento de las operaciones probadas para la gestión de direcciones.

ID	PI-04 — Registrar, consultar y modificar producto
HU asociadas	HU-06, HU-07.
Objetivo	Verificar el funcionamiento del CRUD correspondiente a productos.
Procedimiento	**1.** Registrar un producto con datos válidos. **2.** Consultar el producto. **3.** Modificar uno de sus datos. **4.** Consultar nuevamente la información.
Resultado esperado	El producto se registra, consulta y modifica correctamente.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se registró un nuevo producto con datos válidos desde el frontend, se consultó su información y posteriormente se modificó uno de sus datos. Se verificó que tanto el producto como su actualización se reflejaran correctamente en el sistema y en Supabase cuando correspondía.
Evidencia	`PI-04-01`, `PI-04-02`, `PI-04-03`, `PI-04-04`, `PI-04-05`
Conclusión	**Aprobada**, ya que se verificó el registro, la consulta y la modificación de productos desde el frontend, junto con la persistencia de los datos y sus cambios en Supabase. Las respuestas HTTP obtenidas respaldan el funcionamiento correcto de las operaciones de creación y actualización del módulo de productos.

ID	PI-05 — Consultar existencias
HU asociadas	HU-08.
Objetivo	Verificar que el sistema permita consultar el stock de un producto.
Procedimiento	**1.** Seleccionar un producto con inventario registrado. **2.** Consultar sus existencias. **3.** Comparar el stock mostrado con la cantidad registrada en inventario.
Resultado esperado	El sistema muestra correctamente las existencias disponibles del producto.
Resultado obtenido	La prueba se **ejecutó exitosamente**. En la interfaz de Productos se incorporaron y visualizaron las columnas «Stock Disponible» y «Stock Total», permitiendo consultar las existencias asociadas al producto **PR-001 — Producto con Inventario Asociado**. Se obtuvo un **stock disponible de 20 L** y un **stock total de 30 L**. Estos valores **coincidieron** con las cantidades registradas en los **tres lotes asociados** al producto en la interfaz de Control de lotes y en Supabase: dos lotes de 10 L con estado «Disponible» y uno de 10 L con estado «Reservado».
Evidencia	`PI-05-01`, `PI-05-02`, `PI-05-03`, `PI-05-04`, `PI-05-05`
Conclusión	**Aprobada**, ya que se verificó que el sistema permite consultar las existencias disponibles y totales de un producto, obteniendo valores coherentes con las cantidades y los estados de sus lotes de inventario.

ID	PI-06 — Registrar lote de inventario
HU asociadas	HU-09.
Objetivo	Verificar el registro de información correspondiente a un lote de inventario.
Procedimiento	**1.** Seleccionar un producto. **2.** Registrar los datos del lote. **3.** Guardar el registro. **4.** Consultar el inventario. **5.** Verificar la información almacenada.
Resultado esperado	El lote queda registrado correctamente y puede ser consultado posteriormente.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se seleccionó un producto y se registró un nuevo lote de inventario con sus datos correspondientes. Posteriormente, se verificó que el lote apareciera correctamente en la interfaz de Control de lotes y que la información quedara almacenada en Supabase.
Evidencia	`PI-06-01`, `PI-06-02`, `PI-06-03`, `PI-06-04`, `PI-06-05`
Conclusión	**Aprobada**, ya que se verificó que el sistema permite registrar lotes asociados a productos y consultar posteriormente la información almacenada. Asimismo, se confirmó la persistencia de los datos en Supabase, validando el funcionamiento de la operación de creación de lotes de inventario.

ID	PI-07 — Registrar movimiento de inventario
HU asociadas	HU-10.
Objetivo	Verificar el registro de entradas, salidas o ajustes de inventario.
Procedimiento	**1.** Seleccionar un producto y su inventario/lote respectivo. **2.** Indicar tipo y cantidad **3.** Registrar un movimiento. **4.** Consultar el historial correspondiente.
Resultado esperado	El movimiento queda almacenado correctamente y aparece en el historial.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se seleccionó un producto y su lote de inventario asociado, que contaba con una cantidad inicial de 100 L. A continuación, se registró un movimiento de tipo «Salida» por 20 L. La solicitud se procesó correctamente y el movimiento quedó almacenado en Supabase.
Evidencia	`PI-07-01`, `PI-07-02`, `PI-07-03`, `PI-07-04`, `PI-07-05`
Conclusión	**Aprobada**, ya que se verificó el registro de un movimiento de salida de inventario y su almacenamiento en Supabase. El resultado respalda el funcionamiento de la operación de

	creación de movimientos, así también como el cambio aplicado correspondiente al producto y lote involucrado.

3.2 Usuarios, roles y autenticación

ID	PI-08 — Crear usuario con rol
HU asociadas	HU-11.
Objetivo	Verificar que un usuario pueda ser registrado con uno de los roles disponibles.
Procedimiento	**1.** Ingresar al módulo de usuarios como administrador. **2.** Seleccionar crear usuario. **3.** Ingresar nombre y correo. **4.** Seleccionar un rol. **5.** Guardar. **6.** Consultar el usuario registrado.
Resultado esperado	El usuario se registra correctamente y queda asociado al rol seleccionado.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se ingresó al módulo de usuarios con una cuenta de administrador, se registró un nuevo usuario con sus datos y se le asignó uno de los roles disponibles. Posteriormente, se verificó que el usuario y los datos ingresados se visualizaran correctamente en el frontend y quedaran almacenados en Supabase.
Evidencia	`PI-08-01`, `PI-08-02`, `PI-08-03`, `PI-08-04`
Conclusión	**Aprobada**, ya que se verificó que el sistema permite registrar usuarios desde el módulo correspondiente y asociarlos a un rol disponible. Asimismo, se confirmó la persistencia de los datos en Supabase, validando el funcionamiento de la operación de creación de usuarios con asignación de roles.

ID	PI-09 — Iniciar sesión
HU asociadas	HU-12.
Objetivo	Verificar que un usuario registrado pueda iniciar sesión y acceder al sistema.
Procedimiento	**1.** Ingresar credenciales válidas. **2.** Enviar el formulario de inicio de sesión. **3.** Verificar la respuesta. **4.** Comprobar el acceso a las funcionalidades disponibles.
Resultado esperado	El usuario es autenticado correctamente y accede al sistema de acuerdo con su rol.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se inició sesión con credenciales válidas de un usuario con rol de administrador y, tras enviar el formulario, el sistema redirigió automáticamente al dashboard. Posteriormente, se verificó el acceso a la gestión de usuarios, funcionalidad restringida al rol de administrador.

Evidencia	`PI-09-01`, `PI-09-02`, `PI-09-03`, `PI-09-04`, `PI-09-05`, `PI-09-06`
Conclusión	**Aprobada**, ya que se verificó que un usuario con credenciales válidas puede autenticarse, acceder al sistema y utilizar una funcionalidad habilitada para su rol.

ID	PI-10 — Registrar y consultar proveedor
HU asociadas	HU-26, HU-27.
Objetivo	Verificar que un proveedor pueda ser registrado y posteriormente consultado.
Procedimiento	**1.** Registrar un proveedor con datos válidos. **2.** Guardar la información. **3.** Consultar el listado de proveedores. **4.** Verificar que el proveedor aparezca correctamente.
Resultado esperado	El proveedor queda almacenado y disponible para consulta.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se registró un proveedor con datos válidos desde el frontend y, posteriormente, se consultó el listado para verificar que la información se mostrara correctamente. Además, se confirmó que el registro quedó almacenado en Supabase.
Evidencia	`PI-10-01`, `PI-10-02`, `PI-10-03`
Conclusión	**Aprobada**, ya que se verificó que el sistema permite registrar proveedores y consultar posteriormente la información almacenada. La visualización del proveedor en el frontend y su persistencia en Supabase respaldan el correcto funcionamiento de las operaciones probadas.

ID	PI-11 — Modificar proveedor
HU asociadas	HU-28.
Objetivo	Verificar la modificación de los datos de un proveedor existente.
Procedimiento	**1.** Seleccionar un proveedor. **2.** Modificar uno o más datos. **3.** Guardar. **4.** Consultar nuevamente el proveedor.
Resultado esperado	Los datos modificados quedan almacenados correctamente.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se seleccionó un proveedor existente, se modificaron sus datos y se guardaron los cambios. Posteriormente, se verificó que la información actualizada se reflejara correctamente en el frontend y quedara almacenada en Supabase.

Evidencia	`PI-11-01`, `PI-11-02`, `PI-11-02`, `PI-11-04`
Conclusión	**Aprobada**, ya que se verificó que los datos de un proveedor existente pueden modificarse y consultarse nuevamente, manteniéndose los cambios en Supabase.

ID	PI-12 — Eliminar proveedor
HU asociadas	HU-29.
Objetivo	Verificar que un proveedor pueda ser eliminado según las reglas definidas por el sistema.
Procedimiento	**1.** Seleccionar un proveedor de prueba. **2.** Ejecutar la opción de eliminación. **3.** Confirmar la operación, si corresponde. **4.** Consultar nuevamente el listado.
Resultado esperado	El proveedor seleccionado deja de estar disponible en el listado y la eliminación queda correctamente registrada.
Resultado obtenido	La prueba se **ejecutó exitosamente**. Se seleccionó un proveedor de prueba y se ejecutó la operación de eliminación desde el frontend. Posteriormente, se verificó que el proveedor dejara de aparecer en el listado y que el cambio se reflejara en Supabase.
Evidencia	`PI-12-01`, `PI-12-02`, `PI-12-03`, `PI-12-04`
Conclusión	**Aprobada**, ya que se verificó que el sistema permite eliminar un proveedor de prueba y que este deja de estar disponible en el listado, reflejándose también el cambio en Supabase.

4. Pruebas de rendimiento — PR

Las pruebas de rendimiento del Sprint 2 tienen como objetivo verificar el cumplimiento del RNF-11 — Rendimiento.

El RNF-11 establece que las operaciones principales del sistema deberán responder en un tiempo inferior o igual a **3 segundos**, bajo una carga de hasta **10 usuarios concurrentes**, considerando operaciones de consulta, registro y actualización correspondientes al Módulo 1.

Las pruebas se ejecutarán mediante Locust, herramienta que permite simular usuarios concurrentes y registrar métricas como el tiempo de respuesta, la cantidad de solicitudes procesadas y los errores producidos. Se evaluarán operaciones representativas de los módulos de clientes, productos e inventario y proveedores. Asimismo, se incluirá el inicio de sesión por su relevancia para la autenticación del sistema.

Los resultados obtenidos se compararán con el umbral establecido por el RNF-11, considerando tanto los tiempos de respuesta medidos como la presencia de solicitudes fallidas.

ID	PR-01
Escenario probado	Consulta de clientes
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo
Criterio RNF-11	Tiempo de respuesta ≤ 3 segundos
Tiempo medido	Promedio: 1.088,68 ms (1,09 s). Mínimo: 1.054 ms (1,05 s). Máximo: 1.401 ms (1,40 s).
Resultado	Las 568 solicitudes finalizaron sin errores. El tiempo máximo registrado fue de 1,40 segundos, inferior al límite de 3 segundos establecido por el RNF-11.
Estado	Aprobada

ID	PR-02
Escenario probado	Consulta de productos
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo.
Criterio RNF-11	≤ 3 segundos
Tiempo medido	Promedio: 946,05 ms (0,95 s). Mínimo: 925 ms (0,93 s). Máximo: 1.138 ms (1,14 s). Percentil 95: 980 ms (0,98 s).

Resultado	Se procesaron 608 solicitudes sin errores. Todos los tiempos de respuesta registrados se mantuvieron por debajo de los 3 segundos establecidos por el RNF-11.
Estado	Aprobada

ID	PR-03
Escenario probado	Consulta de proveedores
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo
Criterio RNF-11	Tiempo de respuesta ≤ 3 segundos
Tiempo medido	Promedio: 966,15 ms (0,97 s). Mínimo: 920 ms (0,92 s). Máximo: 1.991 ms (1,99 s).
Resultado	Se procesaron 607 solicitudes sin errores. El tiempo máximo registrado fue de 1,99 segundos, inferior al límite de 3 segundos establecido por el RNF-11.
Estado	Aprobada

ID	PR-04
Escenario probado	Inicio de sesión
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo
Criterio RNF-11	Tiempo de respuesta ≤ 3 segundos
Tiempo medido	Promedio: 957,13 ms (0,96 s). Mínimo: 923 ms (0,92 s). Máximo: 1.933 ms (1,93 s).
Resultado	Se procesaron 602 solicitudes de inicio de sesión sin errores. Todas cumplieron las validaciones del script, incluyendo la respuesta HTTP 200, la presencia de los campos esperados y el estado activo del usuario. El tiempo máximo registrado fue de 1,93 segundos, inferior al límite de 3 segundos establecido por el RNF-11.
Estado	Aprobada

ID	PR-05

Escenario probado	Consulta de inventario
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo.
Criterio RNF-11	≤ 3 segundos
Tiempo medido	Promedio: 947,95 ms (0,95 s). Mínimo: 920 ms (0,92 s). Máximo: 1.946 ms (1,95 s). Percentil 95: 970 ms (0,97 s).
Resultado	Se procesaron 612 solicitudes sin errores. Todos los tiempos de respuesta registrados se mantuvieron por debajo del límite de 3 segundos establecido por el RNF-11.
Estado	Aprobada

ID	PR-06
Escenario probado	Consulta de movimientos de inventario
Carga aplicada	Hasta 10 usuarios concurrentes, con una incorporación de 2 usuarios por segundo.
Criterio RNF-11	≤ 3 segundos
Tiempo medido	Promedio: 944,09 ms (0,94 s). Mínimo: 921 ms (0,92 s). Máximo: 1.113 ms (1,11 s). Percentil 95: 980 ms (0,98 s).
Resultado	Se procesaron 622 solicitudes sin errores. Todos los tiempos de respuesta registrados se mantuvieron por debajo del límite de 3 segundos establecido por el RNF-11.
Estado	Aprobada

Procedimiento

Para cada escenario de rendimiento se seguirá el siguiente procedimiento:

1. Levantar el ambiente dockerizado correspondiente al Sprint 2.
2. Verificar que el backend desarrollado en Django se encuentre operativo y que exista conexión con Supabase.
3. Identificar el endpoint de la API correspondiente a la operación que será evaluada.
4. Configurar Locust para simular hasta 10 usuarios concurrentes, con una tasa de incorporación de 2 usuarios por segundo y una pausa de 1 segundo entre tareas de cada usuario.
5. Ejecutar el escenario de prueba y registrar las estadísticas generadas por Locust.
6. Registrar el número de solicitudes procesadas, las solicitudes fallidas, el tiempo de respuesta promedio, mínimo y máximo, y los percentiles disponibles.
7. Comparar los tiempos de respuesta obtenidos con el límite de 3 segundos establecido en el RNF-11.

8. Conservar los archivos generados durante la ejecución como evidencia, identificándolos mediante la nomenclatura `PR-XX-XX`, de acuerdo con la siguiente clasificación:
    - `PR-XX-01-Charts.png`: captura de los gráficos generados por Locust.
    - `PR-XX-02-Report.html`: reporte detallado de la ejecución.
    - `PR-XX-03-Requests.csv`: estadísticas de las solicitudes procesadas.
    - `PR-XX-04-Exceptions.csv`: registro de excepciones detectadas durante la prueba.
    - `PR-XX-05-Failures.csv`: registro de solicitudes fallidas.
    - `PR-XX-06-Locustfile.py`: script utilizado para configurar y ejecutar el escenario de prueba.
9. En caso de detectar tiempos superiores al límite establecido o solicitudes fallidas, documentar las observaciones para evaluar las posibles causas y las acciones de mejora necesarias.

Criterio de aceptación

Una prueba de rendimiento será considerada **Aprobada** cuando el tiempo de respuesta promedio de la operación evaluada sea menor o igual a **3 segundos**, bajo una carga de hasta **10 usuarios concurrentes**, y no se registren solicitudes fallidas atribuibles al funcionamiento del sistema.

Los tiempos mínimo y máximo, junto con los percentiles obtenidos, se utilizarán como métricas complementarias para analizar la consistencia de las respuestas e identificar posibles demoras puntuales.

Cuando el tiempo promedio supere los 3 segundos o se registren solicitudes fallidas atribuibles al sistema, la prueba será considerada Rechazada y deberá documentarse la observación correspondiente para determinar las acciones de mejora necesarias.

4. Pruebas de seguridad — PS

Debido a que el Sprint 2 incorpora usuarios, roles y autenticación, se realizarán pruebas orientadas principalmente a autenticación, autorización y validación de entradas.

ID	PS-01 — Inicio de sesión con credenciales válidas
Objetivo	Verificar que un usuario registrado pueda autenticarse correctamente.
Procedimiento	**1.** Ingresar correo y contraseña válidos. **2.** Intentar iniciar sesión. **3.** Verificar el acceso al sistema.
Resultado esperado	El usuario accede correctamente al sistema.
Resultado obtenido	El usuario ingresó credenciales válidas y accedió correctamente al sistema, siendo redirigido al dashboard correspondiente.
Evidencia	`PS-01-01`, `PS-01-02`, `PS-01-03`, `PS-01-04`
Conclusión	**Aprobada**. Se verificó que un usuario registrado puede autenticarse correctamente con credenciales válidas.

ID	PS-02 — Inicio de sesión con credenciales inválidas
Objetivo	Verificar que las credenciales incorrectas no permitan el acceso.
Procedimiento	**1.** Ingresar un correo registrado con contraseña incorrecta. **2.** Intentar iniciar sesión.
Resultado esperado	El sistema rechaza la autenticación y no permite acceder a las funcionalidades protegidas.
Resultado obtenido	El sistema **rechazó** el inicio de sesión al ingresar un correo registrado junto con una contraseña incorrecta. El frontend mostró el mensaje «Credenciales inválidas» y la API respondió con **HTTP 401 Unauthorized**, sin conceder acceso al sistema.
Evidencia	`PS-02-01`, `PS-02-02`
Conclusión	**Aprobada**. Se verificó que el sistema rechaza las credenciales incorrectas e impide la autenticación del usuario, conforme al resultado esperado.

ID	PS-03 — Acceso sin autenticación
Objetivo	Verificar que las funcionalidades que requieren autenticación no

	puedan utilizarse sin iniciar sesión.
Procedimiento	1. No iniciar sesión. 2. Intentar acceder directamente a una funcionalidad o endpoint protegido. 3. Registrar la respuesta obtenida.
Resultado esperado	El sistema rechaza el acceso no autenticado.
Resultado obtenido	El backend rechazó la solicitud al endpoint de clientes cuando se intentó acceder sin un token de autenticación, devolviendo el código HTTP 401 Unauthorized.
Evidencia	`PS-03-01`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el sistema impidió el acceso no autenticado al endpoint protegido.

ID	PS-04 — Autorización según rol
Objetivo	Verificar que el acceso a las funcionalidades dependa del rol o permisos asignados al usuario.
Procedimiento	1. Ingresar con un usuario administrador. 2. Registrar las operaciones disponibles. 3. Ingresar con un usuario con privilegios. 4. Ingresar con un usuario base. 5. Intentar realizar una operación que no corresponda al nivel de acceso asignado.
Resultado esperado	Cada usuario puede acceder solamente a las funcionalidades permitidas por su rol.
Resultado obtenido	Las pruebas evidenciaron que el acceso a los endpoints depende de los permisos asignados al usuario autenticado. Las operaciones autorizadas se ejecutaron correctamente, mientras que las no autorizadas fueron rechazadas por el backend.
Evidencia	`PS-04-01`, `PS-04-02`, `PS-04-03`, `PS-04-04`, `PS-04-05`, `PS-04-06`, `PS-04-07`
Conclusión	**Aprobada**. La prueba fue satisfactoria, dado que el sistema aplicó las restricciones de acceso de acuerdo con los roles y permisos configurados.

ID	PS-05 — Validación de entradas
Objetivo	Verificar que el backend rechace datos inválidos.

Procedimiento	1. Intentar registrar un cliente con RUT inválido. 2. Intentar registrar un cliente con correo inválido. 3. Registrar los mensajes y códigos de respuesta.
Resultado esperado	El backend rechaza los datos inválidos y devuelve un error de validación.
Resultado obtenido	El backend rechazó las solicitudes con RUT y correo inválidos, devolviendo errores de validación sin registrar los datos incorrectos.
Evidencia	`PS-05-01`, `PS-05-02`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el backend validó los datos recibidos y evitó el registro de información que no cumple las reglas definidas.

ID	PS-06 — Manejo de credenciales
Objetivo	Verificar que las credenciales de los usuarios no sean expuestas innecesariamente en las respuestas de la API.
Procedimiento	1. Registrar o consultar un usuario. 2. Revisar la respuesta obtenida desde el backend. 3. Verificar si la contraseña aparece en la respuesta.
Resultado esperado	La contraseña no debe aparecer como información expuesta en la respuesta del serializer.
Resultado obtenido	La respuesta del endpoint de usuarios no incluyó el campo de contraseña, evitando la exposición de las credenciales en las respuestas de la API.
Evidencia	`PS-06-01`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el serializer excluyó la contraseña de los datos devueltos al cliente.

ID	PS-07 — Validación de rol inválido
Objetivo	Verificar que el sistema rechace roles que no formen parte de los valores definidos.
Procedimiento	1. Intentar crear o modificar un usuario enviando un rol inexistente. 2. Registrar la respuesta del backend.

Resultado esperado	El backend rechaza el valor e informa que el rol es inválido.
Resultado obtenido	El backend rechazó el rol inexistente enviado en la solicitud y devolvió un error de validación, sin crear el usuario.
Evidencia	`PS-07-01`
Conclusión	**Aprobada**. La prueba fue satisfactoria, dado que el sistema restringió la asignación de roles a los valores definidos.

ID	PS-08 — Validación del token de autenticación
Objetivo	Verificar que el backend rechace tokens inexistentes, inválidos o manipulados.
Procedimiento	1. Enviar una solicitud a un endpoint protegido sin token. 2. Repetir la solicitud con un token ficticio. 3. Modificar un carácter de un token válido y volver a enviar la solicitud. 4. Registrar los códigos de respuesta.
Resultado esperado	El backend rechaza las solicitudes con tokens ausentes o inválidos y no entrega los datos protegidos.
Resultado obtenido	El backend rechazó las solicitudes realizadas con tokens ficticios o manipulados, devolviendo el código HTTP 401 Unauthorized. No se permitió el acceso a los datos protegidos mediante credenciales inválidas.
Evidencia	`PS-08-01`, `PS-08-02`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el sistema validó la autenticidad del token antes de permitir el acceso a los endpoints protegidos, impidiendo solicitudes con tokens inválidos o manipulados.

ID	PS-09 — Resistencia a la manipulación del rol
Objetivo	Verificar que un usuario no pueda obtener privilegios adicionales modificando los encabezados de una solicitud.
Procedimiento	1. Iniciar sesión con un usuario que no tenga permisos administrativos. 2. Enviar una solicitud a `GET /api/permisos/` con su token válido. 3. Repetir la solicitud agregando `X-User-Role: 1`. 4. Comparar las respuestas obtenidas.

Resultado esperado	El backend mantiene las restricciones correspondientes al usuario autenticado, independientemente del valor enviado en `X-User-Role`.
Resultado obtenido	Al realizar solicitudes con un token válido de un usuario sin permisos administrativos, la API rechazó el acceso a la gestión de permisos con el código `HTTP 403 Forbidden`. La modificación del encabezado `X-User-Role` no permitió obtener privilegios adicionales.
Evidencia	`PS-09-01`, `PS-09-02`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el backend determinó los privilegios a partir del usuario autenticado y sus permisos, sin confiar en el rol declarado en los encabezados de la solicitud.

ID	PS-10 — Protección de operaciones no autorizadas
Objetivo	Verificar que un usuario autenticado no pueda ejecutar operaciones para las que no tiene permisos específicos.
Procedimiento	1. Iniciar sesión con un usuario sin permiso para crear clientes. 2. Intentar crear un cliente mediante `POST /api/clientes/`. 3. Registrar la respuesta. 4. Comprobar que el cliente no haya sido creado.
Resultado esperado	El backend rechaza la operación con `403 Forbidden` y no modifica los datos.
Resultado obtenido	El backend rechazó la creación de clientes por parte de un usuario autenticado que no contaba con el permiso requerido, devolviendo el código `HTTP 403 Forbidden`. Se verificó que la operación rechazada no generó un nuevo registro en el sistema.
Evidencia	`PS-10-01`
Conclusión	**Aprobada**. La prueba fue satisfactoria, dado que el sistema aplicó los permisos específicos de cada operación y evitó la ejecución de acciones no autorizadas, manteniendo la integridad de los datos.

ID	PS-11 — Integridad del inventario ante operaciones rechazadas
Objetivo	Verificar que una operación de inventario no autorizada no altere las existencias registradas.

Procedimiento	1. Seleccionar un lote de prueba y registrar su stock inicial. 2. Iniciar sesión con un usuario sin permiso para registrar movimientos. 3. Intentar registrar un movimiento de entrada mediante `POST /api/movimientos/`. 4. Registrar la respuesta. 5. Consultar nuevamente el stock del lote.
Resultado esperado	El backend rechaza la solicitud con `403 Forbidden` y las existencias permanecen sin cambios.
Resultado obtenido	El backend rechazó el intento de registrar un movimiento de inventario por parte de un usuario sin el permiso correspondiente, devolviendo el código HTTP 403 Forbidden. Posteriormente, se verificó que las existencias del lote de prueba permanecieran sin cambios y que la operación rechazada no generara un movimiento nuevo.
Evidencia	`PS-11-01`, `PS-11-02`, `PS-11-03`
Conclusión	**Aprobada**. La prueba fue satisfactoria, ya que el sistema impidió la ejecución de movimientos de inventario no autorizados, preservando la integridad de las existencias y evitando modificaciones derivadas de solicitudes rechazadas.

5. Evidencias

Las evidencias de las pruebas realizadas durante el Sprint 2 se conservarán organizadas según la categoría y el identificador de cada caso de prueba. Cada ficha de prueba contempla la siguiente estructura:

ID → Objetivo → Procedimiento → Resultado esperado → Resultado obtenido → Evidencia → Conclusión.

Las capturas de pantalla y los archivos asociados se identificarán mediante una nomenclatura que permita relacionarlos con los casos de prueba correspondientes. Cuando una misma evidencia abarque varios casos, se indicará expresamente el rango de pruebas cubierto para evitar duplicaciones innecesarias.

Organización y trazabilidad de las evidencias

Las evidencias se vincularán con las fichas de prueba correspondientes mediante sus identificadores. Cuando una captura o archivo respalde varios casos de prueba, se hará referencia a él desde cada ficha pertinente, indicando claramente cuáles casos cubre.

Los archivos fuente se conservarán como respaldo técnico de las pruebas automatizadas, mientras que las capturas, los registros de solicitudes y los informes permitirán documentar los resultados obtenidos durante la ejecución.

La documentación distinguirá entre las pruebas ejecutadas y sus resultados, evitando presentar como realizadas aquellas pruebas que únicamente estén implementadas o planificadas.

6. Matriz de trazabilidad de pruebas

Historia de usuario	Funcionalidad	Pruebas asociadas
HU-01	Registrar clientes	PU-01, PU-02, PI-01, PS-05, PS-10
HU-02	Consultar clientes	PI-01, PR-01, PS-03
HU-03	Modificar clientes	PI-02, PS-05, PS-10
HU-04	Registrar direcciones	PI-03
HU-05	Consultar/modificar direcciones	PI-03
HU-06	Registrar productos	PI-04, PS-03, PS-10
HU-07	Consultar/modificar productos	PI-04, PR-02, PS-03
HU-08	Consultar existencias	PU-17, PU-18, PI-05, PR-05
HU-09	Registrar lotes	PU-16, PI-06, PS-03, PS-10
HU-10	Registrar movimientos	PI-07, PR-06, PS-11
HU-11	Gestionar usuarios y roles	PU-09, PU-10, PU-11, PU-12, PU-13, PI-08, PS-04, PS-06, PS-07, PS-09
HU-12	Iniciar sesión	PI-09, PR-04, PS-01, PS-02, PS-08
HU-26	Registrar proveedores	PU-14, PU-15, PI-10, PS-05, PS-10
HU-27	Consultar proveedores	PI-10, PR-03, PS-03
HU-28	Modificar proveedores	PI-11, PS-05, PS-10
HU-29	Eliminar proveedores	PI-12, PS-10

7. Resumen de resultados del Sprint 2

Tipo	Casos planificados	Ejecutados	Aprobados	Rechazados	Pendientes
PU	18	18	18	0	0
PI	12	12	12	0	0
PR	6	6	6	0	0
PS	11	11	11	0	0
Total	**47**	**47**	**47**	0	0

8. Conclusión del Sprint 2

Durante el Sprint 2 se ejecutaron satisfactoriamente las pruebas definidas para verificar el funcionamiento, la integración, el rendimiento y la seguridad del sistema. En total, se realizaron 47 casos de prueba, distribuidos en 18 pruebas unitarias (PU), 12 pruebas de integración (PI), 6 pruebas de rendimiento (PR) y 11 pruebas de seguridad (PS). De acuerdo con los resultados registrados, los 47 casos fueron aprobados, sin pruebas rechazadas ni pendientes.

Las pruebas unitarias permitieron comprobar el comportamiento de funciones, métodos y validaciones específicas del backend, incluyendo la normalización del RUT, la conversión de estados de clientes y usuarios, la validación de roles y permisos, la validación de estados de proveedores, la generación automática de códigos de lote y el cálculo de las existencias de productos.

Por su parte, las pruebas de integración permitieron verificar el funcionamiento de las principales operaciones contempladas en el alcance del Sprint 2, incluyendo la gestión de clientes y direcciones, productos, existencias, lotes y movimientos de inventario, usuarios y roles, autenticación y proveedores. Estas pruebas permitieron comprobar la interacción entre los componentes involucrados y el comportamiento de las funcionalidades implementadas.

En materia de rendimiento, se ejecutaron los seis escenarios definidos mediante Locust, correspondientes a consultas de clientes, productos, proveedores e inventario, inicio de sesión y consulta de movimientos de inventario. Los resultados y las métricas recopiladas quedaron registrados en sus respectivas evidencias, permitiendo evaluar el comportamiento de los endpoints bajo la carga aplicada. La conformidad con el requisito no funcional RNF-11, que establece un tiempo de respuesta máximo de tres segundos, debe sustentarse en las métricas obtenidas durante estas ejecuciones.

Respecto de la seguridad, las pruebas permitieron verificar los mecanismos de autenticación y autorización, el rechazo de credenciales y tokens inválidos, la validación de datos de entrada, la protección de las credenciales, el rechazo de roles no permitidos y la resistencia a intentos de manipulación de privilegios. Asimismo, se comprobó el bloqueo de

operaciones no autorizadas y la protección de la integridad de las existencias ante intentos de registrar movimientos sin los permisos correspondientes.

En conclusión, los resultados registrados indican que los casos de prueba definidos para el Sprint 2 cumplieron sus respectivos criterios de aceptación. La ejecución de las pruebas permitió obtener evidencia del comportamiento de las funcionalidades evaluadas y de los controles implementados, contribuyendo a la verificación de la calidad, la seguridad y la estabilidad del sistema dentro del alcance establecido. Las evidencias recopiladas proporcionan trazabilidad entre los casos de prueba, los procedimientos ejecutados y los resultados obtenidos, y sirven como respaldo para las siguientes etapas del proyecto.